

LEC26 - Efficiency of Algorithms Continued

Lessons in Efficiency

- Big-Oh describes the behaviour of the time to complete an algorithm as the size of the input grows
 - An algorithm can be good on average, and bad in the worst case
 - Ex Quicksort
- Big-Oh is not good at predicting behaviour on small inputs
- Savings space does not always mean saving time
- More complex algorithms don't always save time either

Recursion

- In the recursive step, assume that each recursive call works correctly
 - Partial tracing

Example

- Let $P(n)$ = "For any nested list object of depth n , `sum_list(obj)` returns the sum of the numbers in `<obj>`"
- We want to know that $\forall n \geq 0, P(n)$
- Reasoning about correctness:
 - For `sum_list`, we traced the function and concluded that
 - $P(0)$ is true
 - $P(1)$ is true if $P(0)$ is true
 - $P(2)$ is true if $P(1)$ and $P(0)$ are true
 - ...
 - We did not trace the "if" parts
 - We already convinced ourselves of them
- Reasoning more formally:
 - If we show that $P(0)$ is true $\forall k \geq 0, P(k + 1)$ is true as long as $P(0), \dots, P(k)$ are all true, then we can conclude that $\forall n \geq 0, P(n)$

Base Cases

- There must always be at least one base case
 - There may be more than one
- Any call to a recursive method must ultimately reach a base case
 - Otherwise, we have "infinite" recursion
- Therefore, in the recursive step, we assume that each recursive call works correctly, as long as the inputs to the recursive call are "smaller" than the original input(s)

Infinite Recursion

- Occurs when a series of recursive calls never reaches a base case
- Every call makes a new recursive call, and this continues forever
- Because each function call requires a new stack frame, infinite recursion can lead to an infinite amount of computer memory consumption

Monday, March 31, 2025

LEC26 - Efficiency of Algorithms Continued

- Python protects against this by limiting the total number of stack frames it allows
 - `RecursionError: maximum recursion depth exceeded`
- You can set the recursion limit via the sys module

```
import sys
sys.setrecursionlimit(5000)
```

- However, oftentimes you shouldn't need to do this, as your code is likely the problem